CSE 3031 Computer Architecture

Homework 2

Spring 2026

Deadline: Midnight of Tuesday, Mar. 3

For all problem-solving questions, you must clearly justify your answer. Responses that do not show the solution process or reasoning will not receive credit.

Problem 1 (15 pts): Compute the effective CPI for MIPS using Figure 1. Assume we have made the following measurements of average CPI for instruction types:

Instructions	Clock Cycles
All ALU instructions	1.0
Load-stores	1.4
Conditional branches	
— Taken	2.0
— Not Taken	1.5
Jumps	1.2

Assume that 60% of the conditional branches are taken and that all instructions in the "other" category of Figure 1 are ALU instructions. Average the instruction frequencies of gap and gcc to obtain the instruction mix.

Problem 2 (30 pts, 10/10/10): For the following, we consider instruction encoding for instruction set architectures.
a. [10] Consider the case of a processor with an instruction length of 12 bits and 32 general-purpose registers, so the size of the address fields is 5 bits. Is it possible to have instruction encodings for the following? Explain your answer.
· 3 two-address instructions
· 30 one-address instructions
· 45 zero-address instructions
b. [10] Assuming the same instruction length and address field sizes as above, determine if it is possible to have instruction encodings for the following and explain your answer.
· 3 two-address instructions
· 31 one-address instructions
· 35 zero-address instructions
c. [10] Assume the same instruction length and address field sizes as above. Further, assume there are already 3 two-address and 24 zero-address instructions. What is the maximum number of one-address instructions that can be encoded for this processor?

Problem 3 (25 pts - 10/15): For the following, assume that values A, B, C, D, E, and F reside in memory. The size of each value is 64 bits. Also, assume that instruction operation codes are represented in 8 bits, memory addresses are 64 bits, and register addresses are 6 bits.
a. [10] For each instruction set architecture shown in Figure 2, how many addresses, or names, appear in each instruction for the code to compute C = A + B, and what is the total code size?
b. [15] Some of the instruction set architectures in Figure 2 destroy operands in the course of computation. This loss of data values from the processor's internal storage has performance consequences. For each architecture in Fig. 2, write the code sequence to compute:
C = A + B
D = A – E
F = C + D
In your code, mark each operand that is destroyed during execution and mark each "overhead" instruction

Instruction	gap	gcc	gzip	mcf	perlbmk	Integer average
load	26.5%	25.1%	20.1%	30.3%	28.7%	26%
store	10.3%	13.2%	5.1%	4.3%	16.2%	10%
add	21.1%	19.0%	26.9%	10.1%	16.7%	19%
sub	1.7%	2.2%	5.1%	3.7%	2.5%	3%
mul	1.4%	0.1%				0%
compare	2.8%	6.1%	6.6%	6.3%	3.8%	5%
load imm	4.8%	2.5%	1.5%	0.1%	1.7%	2%
cond branch	9.3%	12.1%	11.0%	17.5%	10.9%	12%
cond move	0.4%	0.6%	1.1%	0.1%	1.9%	1%
jump	0.8%	0.7%	0.8%	0.7%	1.7%	1%
call	1.6%	0.6%	0.4%	3.2%	1.1%	1%
return	1.6%	0.6%	0.4%	3.2%	1.1%	1%
shift	3.8%	1.1%	2.1%	1.1%	0.5%	2%
AND	4.3%	4.6%	9.4%	0.2%	1.2%	4%
OR	7.9%	8.5%	4.8%	17.6%	8.7%	9%
XOR	1.8%	2.1%	4.4%	1.5%	2.8%	3%
other logical	0.1%	0.4%	0.1%	0.1%	0.3%	0%
load FP						0%
store FP						0%
add FP						0%
sub FP						0%
mul FP						0%
div FP						0%
mov reg-reg FP						0%
compare FP						0%
cond mov FP						0%
other FP						0%

Figure 1: MIPS dynamic instruction mix for five SPECint2000 programs. Note that integer register-register move instructions are included in the OR instruction. Blank entries have the value 0.0%. (**Hint: load imm and cond move are ALU instructions; call and return are jump instructions.**)

that is included just to overcome this loss of data from the processor's internal storage. What is the total code size, the number of bytes of instructions and data moved to or from memory, the number of overhead instructions, and the number of overhead data bytes for each of your code sequences?

Stack	Accumulator	Register (register-memory)	Register (load-store)
`Push A`	`Load A`	`Load R1,A`	`Load R1,A`
`Push B`	`Add B`	`Add R3,R1,B`	`Load R2,B`
`Add`	`Store C`	`Store R3,C`	`Add R3,R1,R2`
`Pop C`			`Store R3,C`

Figure 2: The code sequence for C = A + B for four classes of instruction sets. Note that the Add instruction has implicit operands for stack and accumulator architectures and explicit operands for register architectures. It is assumed that A, B, and C all belong in memory and that the values of A and B cannot be destroyed.

How to submit: Write down your answer in a docx or pdf file. Upload the file through the submission link on Canvas.